TPU FlexAttention: Flexible Attention Kernels on TPUs with JAX + Pallas

Kellen Kanarios Venkatkrishnan Iyer

April 2026

Abstract

We present **TPU FlexAttention**, a Pallas implementation of the FlexAttention API targeting TPU. The kernel exposes the same `score_mod` and `mask_mod` hooks, lowers them into a tiled blockwise-softmax attention kernel that respects the TPU's `(sublane, lane)` 2D vector register layout, and uses the mask function at the grid level to skip fully-masked $(q_{\text{block}}, kv_{\text{block}})$ pairs before kernel launch. Using softcapping as our score function at sequence lengths 1K-16K, we match the hardcoded variant SplashAttention on causal attention, outperforming naive JAX implementation for the forward pass. On the backward pass for large sequence length (16k-65k) we are also able to match SplashAttention. Attention is the dominant compute and memory bottleneck in transformer training and inference, and a growing zoo of attention variants–causal, sliding-window, document-block, ALiBi, PrefixLM, NATTEN, and so on–each demands bespoke kernels to run at hardware peak. PyTorch's FlexAttention [2] addresses this on GPU by exposing a compositional `score_mod` / `mask_mod` API that `torch.compile` lowers into a fused FlashAttention-style Triton kernel, matching hand-written performance while avoiding materialization of the $N \times N$ score matrix. No equivalent exists for TPU: SplashAttention exposes a structured `Mask`-class hierarchy and a small set of score transforms (softcap, logit bias) but not arbitrary `score_mod` / `mask_mod` callables, and Pallas's reference FlashAttention kernel supports neither. Further, the TPU's 2D-tile execution model makes a naive port of the Triton template both incorrect and slow.

1 Introduction

Modern transformer workloads do not use a single attention operation. Causal decoder attention, bidirectional encoder attention, sliding-window attention for long-context models, document-block attention for packed training, relative-position biases like ALiBi, and mixtures such as prefix-LM attention are now all first-class citizens in production codebases. Implemented naively as a pre-softmax modification of the score matrix, each variant forces materialization of the full $[B, H, N, N]$ tensor in HBM and leaves large amounts of compute on the table. Implemented as a hand-written fused kernel, each variant is hundreds to thousands of lines of backend-specific code that must be re-tuned per accelerator generation. Neither path scales with the rate at which new variants appear in the literature.

PyTorch's FlexAttention [2] resolves this tension on GPU. A user writes two small Python functions–a `score_mod(score, b, h, q_idx, kv_idx)` that modifies the pre-softmax logit, and a `mask_mod(b, h, q_idx, kv_idx)` that returns whether a position is attended–and `torch.compile` stamps these into a FlashAttention-style [1] Triton [8] template that (i) tiles Q and K along the sequence dimension to keep the score tile in SRAM, (ii) inlines the user-supplied modification into the template body so no extra kernels are launched, and (iii) uses `mask_mod` at the block level to entirely skip tiles whose index block is fully masked out. The result is an API that recovers hand-tuned FlashAttention performance across a large family of variants from a few lines of high-level Python. TPUs, however, are not GPUs. The TPU's core compute primitive is the MXU, a 128×128 systolic array; its vector registers are natively 2D, with contiguous `(sublane, lane)` layouts, and arithmetic operations act on entire 2D tiles at once. Pallas [4]–the JAX kernel-authoring language that lowers to Mosaic on TPU–inherits this execution model. This divergence from the Triton / CUDA thread-block model changes what a FlexAttention implementation must look like in several material ways:

1. **Tile granularity is fixed by hardware, not by the user.** Q, K, and V blocks must be multiples of the MXU shape, and the `score_mod` and `mask_mod` user functions therefore receive *tiles* of indices rather than scalars. Both must vectorize over the tile shape, not a thread lane.
2. **Pallas / Mosaic has narrower primitive coverage than Triton.** Several operations Triton exposes natively–fine-grained indexing, scalar predication inside a tile, arbitrary `tl.where` over boolean masks–must either be expressed through tile-level Pallas operations or lowered via custom rules. A verbatim port of the Triton template fails to lower.
3. **Block-level mask skipping requires cooperation with the Pallas grid.** On GPU, skipping a

tile is a conditional branch inside the kernel. On TPU, it is most efficiently done by enumerating a sparse list of active $(q_{\text{block}}, kv_{\text{block}})$ pairs on host and iterating only those via the Pallas grid, so that masked blocks are never enqueued as DMAs.

4. **Existing TPU attention kernels expose narrow extensibility points.** SplashAttention [5] accepts user-defined masks through a `Mask` class hierarchy and supports a fixed set of score transforms (softcap, logit bias); its block-sparse scheduler already skips fully-masked blocks. It does not, however, accept free-form `(b, h, q_idx, kv_idx)` mask callables or arbitrary pre-softmax `score_mod` hooks. Pallas's reference FlashAttention [3] supports neither. Naively inlining a user-written modifier into either kernel body breaks its pipelining and register scheduling.

This work closes the gap. We present **TPU FlexAttention**, a Pallas kernel that implements the FlexAttention API on TPU and recovers fused-kernel performance across the standard variant set. Our contributions are: Our contributions are:

- **A Pallas FlexAttention kernel** that accepts `score_mod` and `mask_mod` as traced JAX callables and fuses them into a FlashAttention-style blockwise-softmax loop over K / V tiles. Unlike SplashAttention's `Mask`-class hierarchy, the kernel accepts free-form `(b, h, q_idx, kv_idx)` callables matching the GPU FlexAttention API, and exposes an arbitrary pre-softmax `score_mod` hook rather than the fixed set of score transforms (softcap, logit bias) supported by existing TPU kernels.

- **A block-sparse grid scheduler built on Pallas scalar prefetch.** We evaluate `mask_mod` at block granularity on host, build an index buffer of active $(q_{\text{block}}, kv_{\text{block}})$ pairs, and feed it to the kernel via scalar prefetch so that the Pallas pipeline emitter never issues DMAs for fully-masked blocks. The same index buffer drives the `next_block` computation consumed by each `BlockSpec`'s `index_map`, so partially-masked blocks fall through to an in-kernel predicate path without stalling the pipeline.

- **A memory-and-pipeline schedule for user-supplied modifiers.** Rather than materializing `score_mod` or `mask_mod` outputs in HBM, the kernel broadcasts tile-shaped q / kv index vectors inside VMEM and applies the modifier in place on the score tile. The `BlockSpec` layout for Q, K, V, and O is chosen so that injecting user code inside the inner loop preserves the double-buffered DMA pipeline inherited from FlashAttention and keeps the running softmax state (m_i, ℓ_i) resident in VMEM across the K / V iteration.

2 Background

2.1 Attention and the fusion wall

The textbook scaled dot-product attention over a query $Q \in \mathbb{R}^{N\times d}$, key $K \in \mathbb{R}^{N\times d}$, and value $V \in \mathbb{R}^{N\times d}$ decomposes into four operations:

$$S = QK^\top/\sqrt{d}, \tag{1}$$

$$S' = \texttt{score\_mod}(S), \tag{2}$$

$$P = \text{softmax}(S'), \tag{3}$$

$$O = PV. \tag{4}$$

Written this way in JAX or PyTorch, each step is a distinct high-level operator whose output is the next step's input. The intermediate tensors S, S', and P are each of shape $[B, H, N, N]$. For the sequence lengths now standard in LLM training ($N = 8K$-$128K$), this N^2 tensor exceeds on-chip memory by orders of magnitude, so a non-fused execution writes each intermediate out to HBM and reads it back – three full N^2 round trips per attention call. At long context, attention becomes HBM-bandwidth-bound long before it becomes compute-bound.

A production compiler will try to fuse this pattern away, and on the surface it looks fusible: every consumer here reads exactly what its producer just wrote. But the softmax in Eq. 3 requires a *reduction* over the N key-axis of S' (subtracting the per-row max, then normalizing by the per-row sum of exponentials). That row-wise reduction forces the entire S' row to be resident before any of P is produced, which in turn forces the $QK^\top$ matmul to complete a full row before softmax may begin. Standard producer-consumer fusion cannot span that barrier: a matmul cannot be elementwise-fused with a reduction along its contraction-adjacent axis, and a reduction cannot be fused with a subsequent matmul that consumes its output along the reduced axis. The $N \times N$ materialization is what the dependency structure demands, not a missed optimization.

FlashAttention [1] breaks this barrier algorithmically rather than through compiler heuristics. Instead of computing the full softmax row and then multiplying by V, it processes K and V in tiles along the key axis and maintains, per query row, a running max m_i and running normalizer ℓ_i. When a new K tile produces a larger max, the accumulated output O_i and normalizer are rescaled in place. The score tile stays in on-chip memory, is never written to HBM, and the N^2 intermediate vanishes. The transformation is not one a compiler can derive: it rewrites the semantics of the op into a sequential loop carrying arithmetic state (m_i, ℓ_i, O_i) across iterations, which is outside the functional IR a JAX/XLA or PyTorch graph expresses. A fused attention kernel must therefore be *written*, not compiled, and FlexAttention's contribution on GPU is to generate that written kernel from user-supplied `score_mod` and `mask_mod` callables via a Triton template.

2.2 XLA on TPU

XLA [7] is the compiler that lowers JAX (and TensorFlow, and PyTorch via PyTorch/XLA) programs to TPU. Given a graph of high-level tensor ops, XLA performs operator fusion along elementwise and reduce boundaries, assigns physical memory layouts to every tensor (deciding which logical axis maps to the sublane dimension and which to the lane dimension of the TPU's 2D vector registers), emits HBM $\leftrightarrow$ VMEM DMAs and overlaps them with compute, selects tile sizes for each matmul so that contractions map cleanly onto the MXU's 128×128 systolic array, and schedules MXU and VPU instructions within the resulting basic blocks. For the vast majority of transformer code – embeddings, linear projections, layer norms, elementwise activations – this pipeline produces near-peak utilization from a few lines of JAX and no kernel code whatsoever.

The pipeline's weakness is exactly the case Section 2.1 described. XLA's fuser reasons in terms of producer-consumer locality on a functional graph; it has no rewrite that turns a matmul $\rightarrow$ reduce $\rightarrow$ matmul chain into a loop carrying $(m_i,\ \ell_i,\ O_i)$ across key-tile iterations. Presented with the naive four-line attention, XLA fuses the pre-softmax bias into the matmul epilogue, fuses the post-softmax elementwise work into the PV matmul prologue, and materializes the full $[B, H, N, N]$ score matrix in HBM between the two. This is the behavior users observe when they see attention dominate the trace at long context: it is not an XLA bug, it is the best schedule available under the op-level semantics XLA is given.

2.3 Pallas: what the author writes versus what stays automatic

Pallas [4] is JAX's kernel-authoring language for exactly these cases. A Pallas kernel is a Python function that operates on `Ref`s (pointers into VMEM) rather than on values, invoked via `pl.pallas_call` with a `grid` of iterations and a `BlockSpec` per operand. On TPU, Pallas lowers to Mosaic, which takes the tile-level kernel body through the same backend that XLA uses: register allocation, MXU instruction selection, and VPU scheduling all happen below the Pallas surface. The division of labor between the kernel author and the toolchain is the part of the TPU programming model most often misunderstood, so we make it explicit.

The kernel author is responsible for the *schedule*: the iteration grid, the mapping from grid coordinates to input and output tiles (the `index_map` on each `BlockSpec`), the tile shapes themselves, any loop-carried state that must live across grid iterations, and whether the grid is dense or sparse. The Pallas pipeline emitter uses the `BlockSpec` index maps to prefetch the next iteration's inputs into VMEM and write back the previous iteration's outputs, overlapped with the current iteration's compute, so that DMA latency is hidden behind the MXU. This double-buffered pipeline is automatic *given* the author's schedule, but the schedule itself is not inferred: choosing a Q-tile size that does not divide the sequence length, an `index_map` that disagrees with the data-dependent access pattern, or a block size whose contraction axis is not a multiple of 128 will each produce a correct-but-slow kernel or a lowering failure. Scalar prefetch – a separate `pl.pallas_call` input class that carries small index-valued arrays into the kernel ahead of the main DMA pipeline – is likewise explicit, and is the mechanism by which a Pallas kernel drives a block-sparse grid: the host computes a list of active $(q_{\text{block}}, kv_{\text{block}})$ pairs, passes it via scalar prefetch, and each `BlockSpec`'s `index_map` reads the list to emit the correct next DMA.

Inside a single grid iteration, however, the author writes JAX. Tile-level arithmetic – the $QK^\top$ matmul on the current tiles, the elementwise score modification, the running softmax update, the PV matmul into the output accumulator – is expressed as ordinary `jnp` operations on VMEM-resident arrays. Mosaic allocates vector registers, assigns the `(sublane, lane)` layouts, and schedules MXU and VPU instructions within the tile, exactly as XLA would for the same arithmetic in a non-kernel context. In particular, the 2D register layout that dominates the TPU literature is *not* something the Pallas author controls directly: choosing a $(Q_{\text{block}}, K_{\text{block}})$ of $(128, 128)$ or $(256, 512)$ is a schedule choice the author makes, but the resulting MXU and VPU schedule for the matmul and softmax on that tile is Mosaic's. The author's job is to arrange for the right tiles to be in VMEM at the right grid iteration, with the right loop-carried state, and with the user-supplied `score_mod` and `mask_mod` inlined into the tile-level arithmetic. The rest of this paper describes how TPU FlexAttention makes those schedule choices.

3 Design

This section describes how a user-written `score_mod` becomes a fused modification of the pre-softmax score tile inside our Pallas kernel. The frontend view fixes the API a user writes against; the backend view traces that API through two lowering stages – fusion-value extraction and block-spec derivation – to the injection point inside the flash-attention inner loop.

3.1 Frontend View

A `score_mod` in our API is a pure JAX function of a single argument:

$$\texttt{score\_mod} : \mathbb{R}^{H \times Q \times K} \rightarrow \mathbb{R}^{H \times Q \times K}. \tag{5}$$

It takes the full pre-softmax score tensor and returns a modified one. Positional and per-head dependence enters through Python closures: the user constructs broadcast-shaped index arrays `h_idx : (H, 1, 1)`, `q_idx : (1, Q, 1)`, `kv_idx : (1, 1, K)`, and

any learned parameters such as per-head ALiBi slopes, and references them inside the function. Soft-capping, ALiBi, and relative-position biases are all one-liners in this form. For example, soft-capping becomes `lambda s: jnp.tanh(s / cap) * cap`, and ALiBi with per-head slopes `slopes : (H, 1, 1)` is `lambda s: s + slopes * (kv_idx - q_idx)`.

This is a deliberate departure from PyTorch FlexAttention's `score_mod(score, b, h, q_idx, kv_idx)` scalar-plus-indices signature. The single-argument broadcast form cooperates with the Pallas fuser's block-spec derivation machinery, which handles elementwise and broadcasting operations cleanly but lacks rules for the `lax.gather` that PyTorch-style positional indexing implies when lifted to block shape. The single-argument form structurally routes users toward pre-shaped captures – which is both what the fuser accepts and, empirically, the shape long-context score modifications want anyway.

Masking is expressed separately, via SplashAttention's `Mask` class hierarchy. A user wraps a per-head predicate in a `FlexMask((Q, K), mask_fn)` and combines masks across heads via `MultiHeadMask`. This object is lowered to splash's `MaskInfo` for block-sparse scheduling at grid construction time, so that fully-masked $(q_{\text{block}}, kv_{\text{block}})$ pairs are never enqueued as DMAs. The score_mod and mask paths are independent: score_mod handles additive and multiplicative modifications to the dense portion of the score tile; mask handles sparsity.

3.2 Backend View

What Pallas needs. To fuse a user-supplied modification into the flash-attention inner loop, `pl.pallas_call` needs three pieces of information at construction time:

1. A **tile-level callable** that consumes the current (B_q, B_k) score tile, produces a same-shape result, and contains no Python closures over JAX arrays. Everything the kernel touches must arrive through explicit inputs.

2. The **captured tensor data** – any array the user's `score_mod` closes over (slopes, position arrays, learned biases) – lifted out of the closure and passed as explicit `pallas_call` inputs so the kernel can load them from HBM.

3. A **per-capture block specification**: for each captured tensor, the shape of the slice the kernel consumes on a single $(h, q_{\text{block}}, k_{\text{block}})$ iteration, together with a map from block coordinates to that slice's offset inside the full tensor. Without this the kernel has no principled way to decide which bytes of a captured tensor to read per iteration.

Missing any of the three and the fusion cannot proceed: no callable means no arithmetic to inline, no captured data means the values are not in the kernel's address space, and no block specification means the kernel cannot localize the read to the current block.

How `_tile_score_mod` supplies them. All three come from a single host-side trace of the user's `score_mod` through JAX's Pallas fuser, wrapped in a helper we call `_tile_score_mod`. It runs the user function once on a `ShapeDtypeStruct` of the full (H, Q, K) score tensor. The fuser returns a rewritten function whose closures have been hoisted into an explicit input pytree, together with the captured tensor values extracted from those closures. We then hand the rewritten function a desired output block specification of (B_q, B_k) over the sequence axes, and the fuser walks the jaxpr backward, deriving for every captured value a matching `pl.BlockSpec`. Under this machinery, a broadcast-shaped ALiBi slope array $(H, 1, 1)$ comes out with block shape $(\texttt{None}, 1, 1)$; a kv-indexed position array $(1, 1, K)$ comes out with block shape $(\texttt{None}, 1, B_k)$; a pure positional ALiBi body that closes over nothing produces an empty captured-values pytree. The three return values line up exactly with Pallas's three requirements: rewritten function, captured values, derived block specifications.

Integration caveat. One detail prevents handing the derived block specifications straight to `pl.pallas_call` as `in_specs`: Pallas can only grid-slice inputs along axes that are iteration dimensions of the `pallas_call` grid. Our kernel grid runs over (H, Q_{block}), but not over K_{block}. The K-block is an *inner* loop, because flash-attention's running softmax state (m_i, ℓ_i, O_i) must persist across K-iterations within a single Q-block, and promoting K to a grid axis would force that state to round-trip through HBM on every iteration – negating the entire premise of flash attention. Captured tensors whose block specifications depend on the K-index therefore cannot be grid-sliced. We pass all captures in as whole arrays and apply each capture's block specification manually inside the inner K loop, reading the index map to decide which slice to load. The derived specifications are exactly the right objects for this, regardless of whether they end up consumed by Pallas's grid machinery or by our inner-loop logic.

Where the call sits. The rewritten `score_mod` runs between the $QK^\top$ matmul and the mask application, on the current (B_q, B_k) score tile. This order is load-bearing: soft-cap and additive bias need to see raw logits (so `score_mod` must precede masking), and the mask must write $-\infty$ *after* `score_mod` so the mask sentinel is not overwritten by subsequent arithmetic. Prior to adding `score_mod`, soft-capping was a separate kernel option with its own parameter; it is now expressible as a one-line score modification with no kernel-side changes.

Backward pass. We register the forward kernel through `jax.custom_vjp` and reuse SplashAttention's

block-structured `dq` and `dkv` backward kernels, with one change: where the original kernels carried a hand-derived jacobian specific to soft-cap (multiplying the incoming cotangent by $1 - \tanh^2(qk/c)$), we replace that line with the VJP of the user's `score_mod`, captured at the same (B_q, B_k) granularity as the block being processed. Concretely, each backward kernel computes $qk = qk^\top$ exactly as the forward does, then evaluates

$$(\tilde{qk},\ f_{\mathrm{VJP}}) = \texttt{jax.vjp}(\texttt{score\_mod},\ qk) \tag{6}$$

on the current block. The existing FlashAttention-style gradient assembly runs forward to $\partial L/\partial \tilde{qk}$ in the usual way ($(\mathrm{dp} - \mathrm{di}) \odot p$), and we then apply f_{VJP} once to push that through `score_mod` to obtain $\partial L/\partial qk$, which the existing $dq = ds \cdot k$ and $dk = ds^\top \cdot q$ matmuls consume unchanged.

This gives us correct gradients for arbitrary `score_mod` – including the captured tensors it closes over – with no per-score-mod jacobian derivation, while keeping the backward fully block-structured: no N^2 attention matrix is materialized in the backward any more than in the forward, and both `dq` and `dkv` still run at splash's block-sparse scheduling granularity. The only new memory cost is that of storing f_{VJP}'s linearization state for the duration of a single block's backward computation, which is the same footprint as the forward's score tile.

4 Methodology

4.1 Experimental Setup

We benchmark on a single configuration – **causal mask** combined with **soft-cap** ($\tanh(s/c) \cdot c$) as the score modification – because it is the one attention variant SplashAttention already supports as a dedicated kernel option. This lets us measure our implementation against a hand-tuned TPU baseline on exactly matched work, isolating the cost of going through the flexible `score_mod` path rather than a single-variant kernel.

For the forward pass we compare three implementations:

1. **Flex** – our implementation, with soft-cap expressed as the score_mod `lambda s: jnp.tanh(s / c) * c` and causal masking via a `CausalMask`.

2. **Splash** – SplashAttention with its dedicated `attn_logits_soft_cap` kwarg and the same causal mask. This is the TPU-native baseline.

3. **Naive JAX** – an unfused reference (`einsum` → `tanh` → `where` → `softmax` → `einsum`) that materializes the full $N \times N$ attention matrix in HBM between steps.

For the backward pass we drop the naive JAX reference, as it runs out of memory at all tested sequence lengths. The remaining comparison is Flex (our custom `jax.vjp`-per-block backward) against Splash's fully hand-written dq/dkv kernels with the soft-cap-specific jacobian inlined.

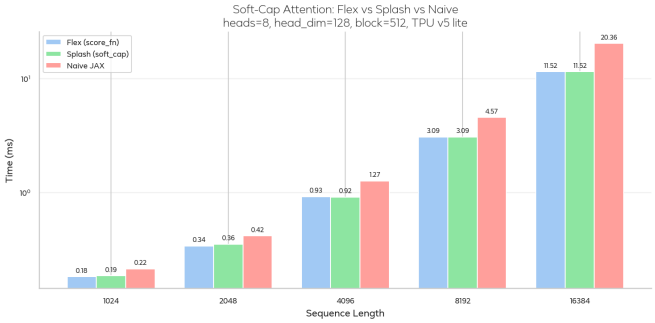

Figure 1: Forward soft-cap causal attention: Flex (ours) vs. Splash vs. naive JAX.

5 Results

5.1 Forward Performance

Figure 1 compares Flex, Splash, and naive JAX. Flex and Splash track each other closely across all tested sequence lengths, and both pull away from naive JAX as sequence length grows – exactly the regime in which fused execution and careful HBM traffic matter most.

The important result is the Flex-Splash gap: it is small and does not grow with sequence length. Since the two kernels differ only in how soft-cap reaches the kernel body – a dedicated kwarg vs. a generic `score_mod` hook with fuser-derived block specs – this pins the cost of flexibility itself. For this configuration, that cost is essentially zero.

5.2 Backward Performance

Figure 2 compares Flex and Splash on the backward pass. Again the two implementations track closely across all tested sequence lengths.

This is the stronger result of the two. Flex's backward replaces Splash's hand-derived soft-cap jacobian $\partial_s(\tanh(s/c) \cdot c) = 1 - \tanh^2(s/c)$ with a generic `jax.vjp(score_mod, qk)` captured once per block. Matching Splash here means the VJP trace produces a block-local linearization whose execution cost is indistinguishable from the hard-coded version – which is the non-obvious outcome. If it were slower, it would indicate that `jax.vjp` is introducing overhead that kills the approach for any non-trivial `score_mod`; since it isn't, we have evidence that the same construction will carry over to ALiBi, relative-position biases, and arbitrary user modifications without paying a backward-pass tax.

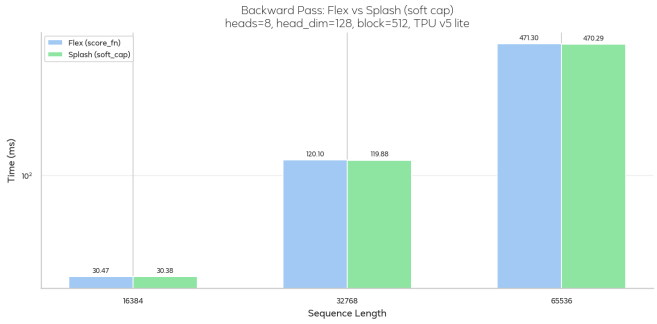

Figure 2: Backward soft-cap causal attention: Flex (ours) vs. Splash.

5.3 Interpretation

The single configuration we benchmark – causal + soft-cap – is not representative of the full attention variant zoo, but it is the one configuration where a direct Flex-vs-Splash comparison is possible with matched work on both sides. On that comparison, Flex matches Splash in both directions, which isolates two claims:

- The fuser-derived `score_mod` path imposes no measurable forward-pass overhead relative to a dedicated kernel kwarg. The broadcast-shaped captures and per-block value slicing described in Section 3.2 cost nothing that Splash's hand-specialized path saves.
- Replacing Splash's hand-derived soft-cap backward jacobian with a generic `jax.vjp(score_mod, qk)` captured per block does not slow the backward down. This is the generalizing claim of the work – the same machinery applies unchanged to any future `score_mod` the user writes.

Both claims are conditional on a `score_mod` the fuser can lower. The failure modes we encountered during development (`exp2` lacking a `pull_block_spec` rule, position-indexed gathers not being tileable) remain as pitfalls. What this benchmark establishes is that when the fuser path succeeds, it succeeds without a performance penalty worth engineering around.

6 Discussion and Future Work

This work targets the *training* regime: long sequences, full Q and K/V tensors materialized in HBM, and a $(Q_{\text{block}}, K_{\text{block}})$ schedule chosen to keep the MXU saturated across the K/V iteration. The `score_mod` / `mask_mod` API extends to inference unchanged; the kernel schedule around it does not.

Low-batch decode. At $B \ll H$ with Q of length 1, the $QK^\top$ and PV matmuls collapse to HBM-bandwidth-bound operations and the MXU sits idle under the training schedule. A decode kernel wants to pack the batch's queries into a single MXU-shaped tile and minimize bytes-per-output-token on K/V streaming, with `score_mod` / `mask_mod` evaluated once per query row.

Paged KV caches. Real serving stacks allocate the cache in fixed-size pages (PagedAttention [6]) with per-sequence page tables. Mapping this onto Pallas is structurally similar to the block-sparse scheduler developed here: enumerate (sequence, logical page) pairs and drive each `BlockSpec`'s `index_map` through the page table via scalar prefetch.

Sharded KV caches. At $N \geq 128K$, the cache exceeds a single chip's HBM. Head-sharding composes trivially with our kernel, but degrades under MQA/GQA. Sequence-sharding requires combining partial softmax statistics (m_i, ℓ_i, O_i) across chips – the same FlashAttention composition lifted one level up – implementable either at the `shard_map` level or via in-kernel `all_gather` on K/V tiles.

7 Conclusion

We present a TPU implementation of the FlexAttention API that matches SplashAttention's hand-specialized soft-cap performance on both forward and backward passes while admitting arbitrary user-supplied `score_mod` callables. The backward replaces Splash's per-kernel hand-derived jacobian with a per-block `jax.vjp` of the user's `score_mod`, preserving block-sparse scheduling while generalizing to any future score modification the fuser can lower. Extending the same API to the inference regime – decode, paged caches, sequence-sharded caches – is the natural next step: the hooks carry over, only the kernel schedule changes.

8 Team Member Contributions

Kellen Kanarios: Worked on poster/report writing, implementation details, TPU-side experimentation, and evaluation of the flexible attention prototype against TPU baselines.

Venkatkrishnan Iyer: Worked on project framing, poster/report writing, benchmark analysis, and the connection between the FlexAttention paper and TPU-side system design.

References

[1] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems*, 35:16344–16359, 2022.

[2] J. Dong, B. Feng, D. Guessous, Y. Liang, and H. He. Flex attention: A programming model for generating optimized attention kernels. *arXiv preprint arXiv:2412.05496*, 2(3):4, 2024.

[3] JAX Team. Flash Attention TPU kernel (Pallas). `https://github.com/jax-ml/jax/blob/main/jax/experimental/pallas/ops/tpu/flash_attention.py`, 2024. Accessed: 2026-04-24.

[4] JAX Team. Pallas: a JAX kernel language. `https://docs.jax.dev/en/latest/pallas/index.html`, 2024. Accessed: 2026-04-24.

[5] JAX Team. Splash (Sparse flash) Attention TPU kernel (Pallas). `https://github.com/jax-ml/jax/blob/main/jax/experimental/pallas/ops/tpu/splash_attention/splash_attention_kernel.py`, 2024. Accessed: 2026-04-24.

[6] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626, 2023.

[7] The XLA Authors. XLA: Optimizing compiler for machine learning. `https://github.com/openxla/xla`, 2023. Accessed: 2026-04-24.

[8] P. Tillet, H. T. Kung, and D. Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*, pages 10–19, New York, NY, USA, 2019. Association for Computing Machinery.